

Sport Coaching Tool — stack & integration plan (v2)

Updated version of the original recommendation: backend swapped to NestJS (better fit for a 6-person team), testing swapped to Jest 30. Each layer below has what to use, why we need it per the brief, and how it actually connects to the rest of the system.

Quick reference

Layer	Tool	Talks to
Frontend	React + Vite + TypeScript, Tailwind + shadcn/ui	Backend API (REST, over HTTPS), Socket.io (WebSocket)
Backend & API	NestJS + TypeScript + <code>nestjs-zod</code>	Postgres (via Drizzle), Auth, Socket.io, external APIs
API docs	<code>@nestjs/swagger</code>	Reads NestJS DTOs directly
Auth	better-auth	Backend (as a NestJS module), Postgres (sessions/users table)
Database	PostgreSQL on Neon	Backend only, via Drizzle
ORM	Drizzle	Backend ↔ Neon
Offline client store	SQLite via PowerSync	Frontend device ↔ Neon (background sync)
Real-time	Socket.io	Frontend ↔ Backend, same Node process as NestJS
Weather	OpenWeatherMap	Backend calls it server-side, caches result
Maps/geocoding	Mapbox	Frontend (rendering) + Backend (geocoding on save)
Testing — unit/integration	Jest 30 + Supertest	Runs against backend modules and live-but-local API
Testing — E2E	Playwright	Drives frontend in a real browser, hits real (test) API
CI/CD	GitHub Actions	Runs Jest + Playwright + lint on every PR, deploys on merge
Docs site	Docusaurus → GitHub Pages	Separate static site, not connected to the app
Frontend hosting	Vercel	Deploys from GitHub, calls backend's public URL
Backend hosting	Railway	Connects to Neon over a connection string

Frontend — React + Vite + TypeScript, Tailwind + shadcn/ui

Why: the brief requires a non-monolithic front-end and back-end, and specifically warns off frameworks that fuse the two (Next.js, SvelteKit server routes) — you can only use those for one side. React + Vite is client-only by construction, so there's no risk of accidentally building API routes inside it. Tailwind + shadcn/ui gets us most of the way to "modern responsiveness and accessibility" for free, since shadcn's components are built on accessible primitives (keyboard nav, ARIA) rather than us building that from scratch.

Integration: talks to the backend two ways — normal REST calls (via `fetch`/TanStack Query) for anything that isn't live, like editing a roster or pulling season stats, and a Socket.io connection for the live event dashboard. It never talks to Postgres, Neon, or PowerSync directly — everything server-side is reached through the NestJS API only.

Backend & API — NestJS + TypeScript + `nestjs-zod`

Why: satisfies **Hand-Written API** directly — every route is ours, nothing auto-generated. With 6 people, NestJS's enforced module → controller → service structure matters more than raw speed: it gives everyone the same shape to build in, which cuts down on PR-review friction over "why did you structure this differently." `nestjs-zod` lets us keep a single Zod schema per resource that both validates incoming requests and generates the TypeScript type, so the shape of e.g. an event-log entry is defined exactly once.

Integration: sits in the middle of everything. It's the only thing that talks to Postgres (via Drizzle), the only thing that calls OpenWeatherMap/Mapbox, and it hosts the Socket.io server for real-time. Suggested module split for a team of 6 (two people per module, vertical slice — API + relevant UI together):

- `auth` — signup/signin/roles
- `roster` — athlete and team management
- `events` — the live logging core (this is the one with real-time + offline sync attached)
- `stats` — season totals, comparisons, derived from the event log

API docs — `@nestjs/swagger`

Why: covers the Milestone 2 rubric line "API available externally with documentation" (15% of that milestone).

Integration: reads decorators straight off our NestJS controllers/DTOs and serves an interactive Swagger UI at a route on the backend itself — no separate service, no manual doc-writing to keep in sync.

Auth — better-auth

Why: the brief explicitly forbids writing your own auth system — must use an established library. better-auth is self-hosted (no external vendor dependency for something this core) and covers signup, signin, password reset, and account deletion out of the box, which are all explicitly required.

Integration: runs as a module inside the NestJS app, backed by its own tables in the same Postgres database (via Drizzle). Issues a session cookie/token the frontend attaches to every request; NestJS guards check it on protected routes. Roles (coach vs assistant) live in the same users table and get checked in the same guards.

Database — PostgreSQL on Neon

Why: multiple coaches and assistants need concurrent writes, roles/permissions checked centrally, and relational joins for season stats and athlete comparisons — a client-server database, not an embedded one, is required for this part.

Integration: only the backend ever connects to it, over a connection string, via Drizzle. Note Neon only hosts the database — it doesn't run our backend or frontend code, those still need Railway/Vercel. It also doesn't help with offline — Neon is a cloud service reachable only when a device has signal, which is exactly why the offline piece below exists separately.

ORM — Drizzle

Why: gives compile-time safety on database queries (typo a column name, get a build error instead of a runtime crash) and one schema definition that both defines the table shape and generates migrations via `drizzle-kit`.

Integration: lives entirely inside the backend. Every NestJS service that needs data goes through Drizzle rather than writing raw SQL strings; `drizzle-kit` diffs our schema against Neon and generates the migration to run in CI.

Offline client store — SQLite via PowerSync

Why: this is the Intermediate/Advanced-tier requirement that logging must work with no signal, and at Advanced tier, that several assistants can log the same event offline and have it merge without conflict — the hardest single requirement in the brief.

Integration: PowerSync runs an embedded SQLite database on each device (inside the frontend app) and handles bidirectional sync with our Neon Postgres in the background once connectivity returns, including conflict resolution — we don't hand-roll merge logic. Only the `events` module needs this; roster edits and stats can assume connectivity.

Real-time — Socket.io

Why: the live event dashboard needs updates to appear the instant an assistant logs something, for anyone else currently watching — plain HTTP polling would mean visible lag and wasted requests.

Integration: runs on the same Node process as the NestJS backend, attached at startup. Frontend opens a socket connection per event ("room"), so an update broadcasts only to people watching that specific event. Distinct from the offline path above — this is for online users watching in real time, PowerSync is for offline capture.

External integrations — OpenWeatherMap + Mapbox

Why: covers the **Integration** key requirement directly (the brief's own example is a weather check for an event's location).

Integration: both are called server-side from the backend, not the frontend — keeps API keys off the client and lets us cache responses (no need to re-fetch weather every time someone views the same event). Mapbox is used twice: geocoding a venue address to coordinates when an event is created, and rendering a small map on the frontend using those coordinates.

Testing — Jest 30 (unit/integration) + Supertest, Playwright (E2E)

Why: feeds the Sprint 2 rubric line "UI and API testing implemented" (10%) and the testing-documentation lines in Sprint 3/4. Jest 30 is NestJS's default/most-integrated test runner (Nest's own testing utilities and `@nestjs/testing` are built assuming Jest), so it's the path of least friction for mocking services and testing modules in isolation — worth noting Vitest is generally faster and more ESM-native for greenfield Vite projects, but since our backend is NestJS rather than Vite, Jest's tighter Nest integration wins here.

Integration: Jest + Supertest test the backend directly — spinning up the NestJS app in-memory and hitting its routes without a real network hop, checking things like "does logging a goal actually update the athlete's stats." Playwright runs separately, driving an actual browser against a real (test) deployment of both frontend and backend together — this is the only layer that can catch things like "does a goal logged in one browser tab actually appear in another tab via the Socket.io connection."

CI/CD — GitHub Actions

Why: covers the CI/CD key requirement directly.

Integration: on every PR, runs lint + Jest + Playwright against a Neon branch created just for that PR (cheap thanks to copy-on-write branching), then tears it down. On merge to main, deploys backend to Railway and frontend to Vercel.

Docs site — Docusaurus → GitHub Pages

Why: satisfies the Documentation Site requirement and "avoid platforms that require an account."

Integration: intentionally not connected to the running app at all — it's a separate static site, written in markdown, deployed independently via its own GitHub Actions job.

Hosting — Vercel (frontend), Railway (backend)

Why: neither Neon nor any database host runs application code, so we need somewhere that actually executes the NestJS process and serves the built frontend.

Integration: Vercel builds and serves the Vite output, and is configured with the backend's public URL as an environment variable so its `fetch` calls know where to go. Railway runs

the NestJS process and holds the Neon connection string as a secret env var — it's the only piece of infrastructure with direct network access to the database.

AI tooling note

Whatever combination of Claude Code, Antigravity, or skill packs (e.g. the Matt Pocock TDD/`grill-with-docs` skills) gets used building any of this, remember the course AI policy requires attribution: `Assisted-by: <tool>[<model>]` in commit messages, plus a running declaration in the README for code generation, in-line editing, and code review tools — including "we don't use X" for anything deliberately skipped.